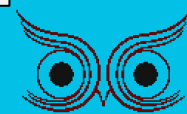
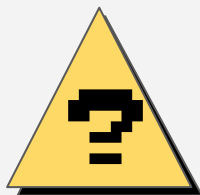




COMPETEC



COMPETEC



Revisão

☐

Don't show this message again

Ok



Assuntos da aula

- > Como o computador funciona?
- > Erros
- > Assuntos passados

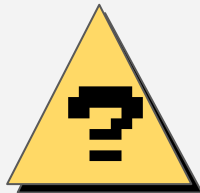
Ok

Cancel

Options<<



COMPETEC



Como o computador
funciona?

☐

Don't show this message again

Ok



O computador funciona com 0s e 1s, mas nós entendemos português.

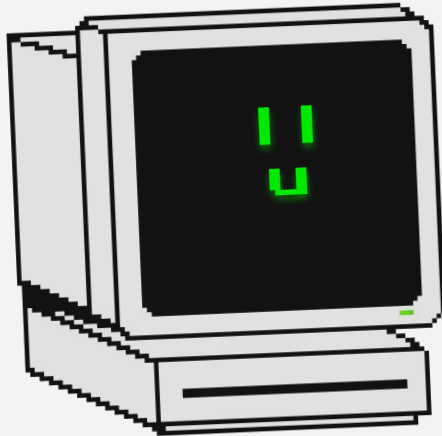
Para programadores conseguirem se comunicar com computadores, precisamos de um “meio termo”: a linguagem de programação!



Como o computador funciona



01100010 01101001
01101110 01100001
01110010 01101001
01101111



Calcule $20+45$

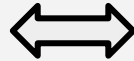
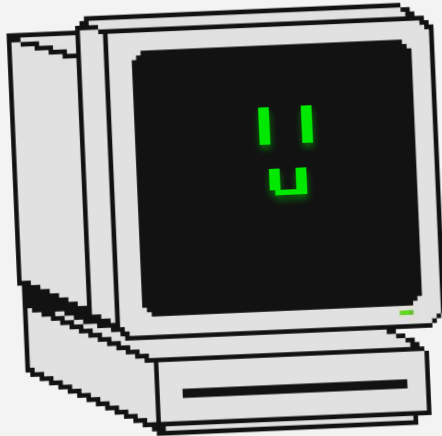


Como o computador funciona

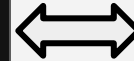


01100010 01101001
01101110 01100001
01110010 01101001
01101111

Calcule $20+45$



```
int soma = 20+45;
```



Mas como o computador entende a linguagem de programação? O compilador!!

O compilador é um “tradutor” da linguagem de programação para os 0s e 1s que o computador entende.

Sempre que executamos (“rodamos”) o código, o compilador:

- Verifica erros no código
- Traduz o código para a língua do computador



O que está errado?

Main.java



```
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println("Hello World")
6     }
7 }
8
```

```
Main.java  ⋮
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println("Hello World")
6     }
7 }
8
```

input

Compilation failed due to following error(s).

```
Main.java:5: error: ';' expected
        System.out.println("Hello World")
                               ^
1 error
```

Erros



```
Main.java
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println("Hello World")
6     }
7 }
```

input

Compilation failed due to following error(s).

Main.java:5: error: ';' expected
System.out.println("Hello World")
^

1 error

Onde o compilador encontra o erro

Erros



```
Main.java
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println("Hello World")
6     }
7 }
```

Onde o compilador encontra o erro

Compilation failed due to following error(s).

Main.java:5: error: ';' expected

System.out.println("Hello World")

Erro encontrado pelo compilador

1 error

Erros



```
Main.java
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println("Hello World")
6     }
7 }
```

Onde o compilador encontra o erro

Compilation failed due to following error(s).

Main.java:5: error: ';' expected

Erro encontrado pelo compilador

Quantidade total de erros encontrados

1 error

O que está errado?

Main.java



```
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println("Hello World");
6     }
7 }
```

O que está errado?

Main.java

```
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         system.out.println("Hello World");
6     }
7 }
```

input

Compilation failed due to following error(s).

```
Main.java:5: error: package system does not exist
        system.out.println("Hello World");
            ^
```

1 error

Como o compilador sabe o que está errado?

Todo idioma é um monte de palavras que possuem sentido, organizadas de uma forma específica. Linguagem de programação também!!

Uma linguagem de programação tem uma lista de *palavras-chave*, que o compilador reconhece:

- exemplos: `if` `int` `System.out.println`

A linguagem de programação, assim como qualquer idioma, possui também pontuação, em que cada símbolo tem uma finalidade específica:

- exemplos: `{ }` `=` `;` `( )`

Sempre que escrevemos coisas fora dessa lista, **precisamos** usar aspas (“ e ”)

0 que está errado?

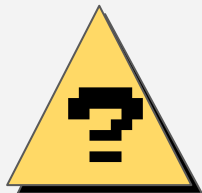
```
Main.java :
1
2 public class Main
3 {
4     public static void main(String[] args) {
5         System.out.println(Hello World);
6     }
7 }
```

input

Compilation failed due to following error(s).

```
Main.java:5: error: ')' or ',' expected
        System.out.println(Hello World);
                           ^
Main.java:5: error: not a statement
        System.out.println(Hello World);
                           ^
Main.java:5: error: ';' expected
        System.out.println(Hello World);
                           ^
3 errors
```

COMPETEC



Curiosidade!!

☐

Don't show this message again

Ok



Você sabia que dá para escrever todo o código na mesma linha?



```
Main.java :
1 public class Main
2 {
3     public static void main(String[] args) {
4         int media = 5;
5         if(media>=5){
6             System.out.println("Você passou!! :)");
7         }
8         else {
9             System.out.println("Está de recuperação :(");
10        }
11    }
12 }
```

input

Você passou!! :)

Você sabia que dá para escrever todo o código na mesma linha?



```
Main.java  ⋮
1  public class Main
2  {
3      public static void main(String[] args) {
4          int media = 5;
5          if(media>=5){
6              System.out.println("Você passou!! :)");
7          }
8          else {
9              System.out.println("Está de recuperação :(");
10         }
11     }
12 }
```

input

Você passou!! :)

```
Main.java  ⋮
1  public class Main{public static void main(String[] args){int media = 5; if(media>=5){
2  System.out.println("Você passou!! :)");}else{System.out.println("Está de recuperação :(");}}}
3
```

input

Você passou!! :)

Então por que deixar recuo?

Onde está o erro?

```
Main.java  ⋮
1 public class Main
2 {
3     public static void main(String[] args) {
4         int media = 5;
5         if(media>=5){
6             System.out.println("Você passou!! :)");
7         }
8         else {
9             System.out.println("Está de recuperação :(");
10        }
11    }

```

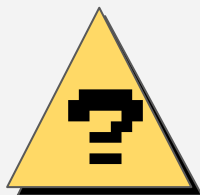
input

Compilation failed due to following error(s).

```
Main.java:11: error: reached end of file while parsing
}
^
1 error

```

COMPETEC



Assuntos passados

- >Variáveis
- >Operações Aritméticas
- >Condicionais (if/else)
- >Loops (while/do while/for)

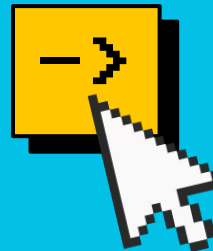
Ok





Aula nº01

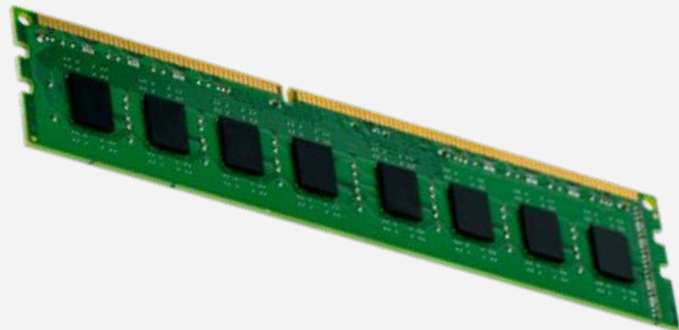
Variáveis



É um espaço na memória do computador, onde podemos armazenar valores;

tipo nomeDaVariavel = valor;

- Tipos: int, long, float/double, char, boolean
- Nomes de variáveis: *não pode ter espaços, não pode começar com um número e não pode ter caracteres especiais/acentos*
- Valor: só precisa ser correspondente ao tipo da variável



Variáveis



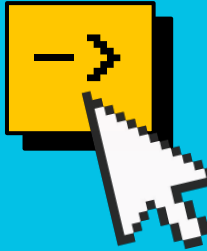
variável camisa

variável sapato



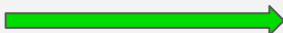
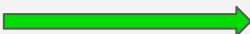
Aula nº01

Operações aritméticas



Conjunto de ações matemáticas que podemos realizar sobre as variáveis.

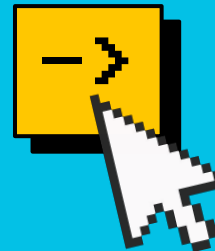
```
int x;  
int y;
```

- Soma (+)  `int soma = x+y;` //x++ é a mesma coisa que `x=x+1`
- Subtração (-)  `int subtracao = x-y;` //x-- é a mesma coisa que `x=x-1`
- Multiplicação (*)  `int multiplicacao = x*y;`
- Divisão (/)  `int divisao = x/y;`
- Resto da divisão (%)  `int resto = x%y;`



Aula nº01

Impressão e
Scanner



Para imprimir algo na tela:

- `System.out.println("texto que quero exibir");`
- `System.out.print("texto" + nomeDeVariavel + "\n");`

Para ler algo da tela:

- Importar a classe Scanner no começo do código
- Criar um Scanner
- Usar o método certo dependendo do tipo do valor a ser recebido

*Lembrem sempre de guardar o valor em uma variável (como no exemplo ao lado)

```
import java.util.Scanner;

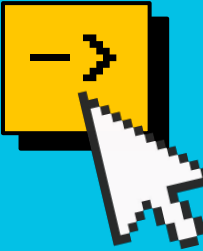
Scanner leitor = new Scanner(System.in);

int numero = leitor.nextInt();
double decimal1 = leitor.nextDouble();
float decimal2 = leitor.nextFloat();
boolean simOuNao = leitor.nextBoolean();
char letra = leitor.nextChar();
String palavra = leitor.next();
String linha = leitor.nextLine();
```

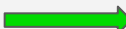
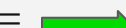
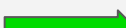
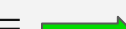
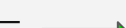
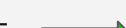


Aula nº02

Condicional
(if/else)



Comparações:

- `>`  maior que
- `>=`  maior ou igual a
- `<`  menor que
- `<=`  menor ou igual a
- `==`  igual a
- `!=`  diferente de

- `&&`  “e”
- `||`  “ou”

// detalhe: ! torna negativa qualquer expressão que vier imediatamente depois

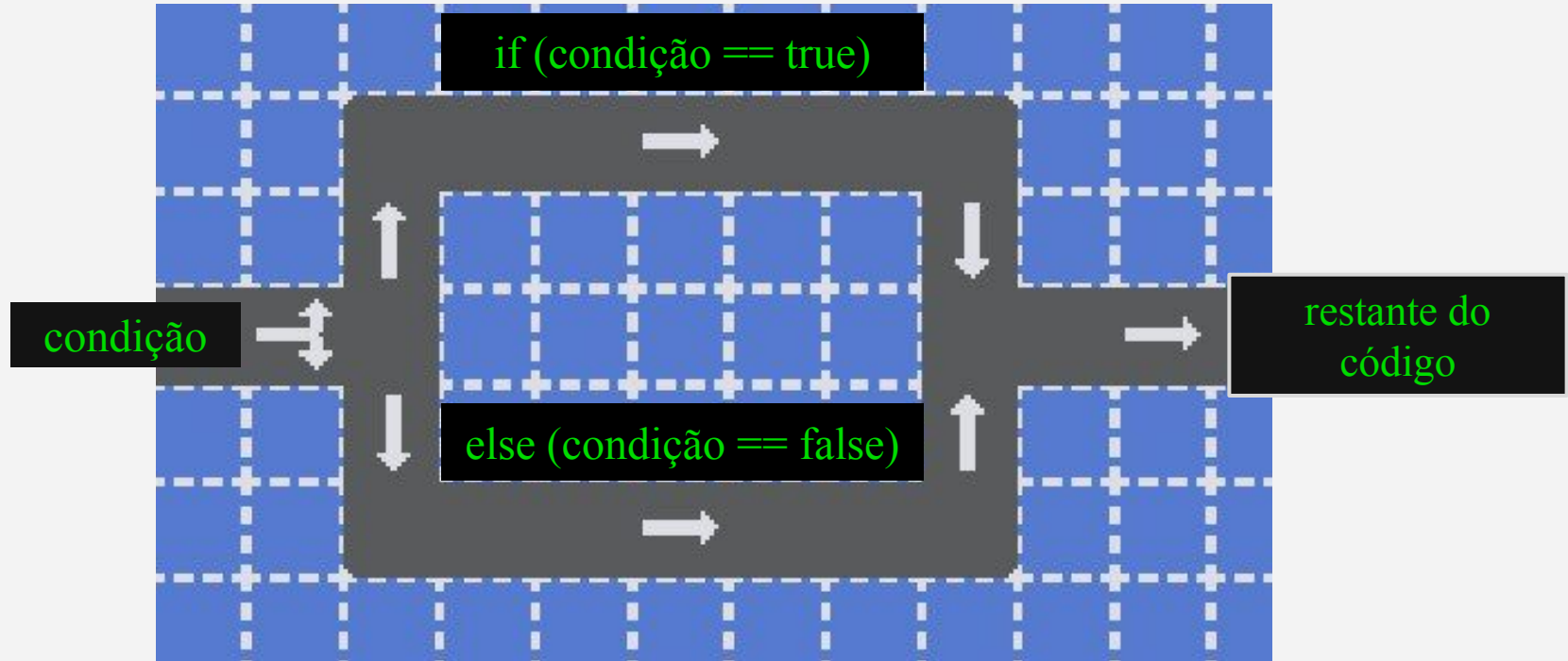
```
boolean variavelSim = true;  
boolean variavelNao = !sim;
```

```
System.out.println(variavelSim); // true  
System.out.println(variavelNao); // false
```

If e else



If e else

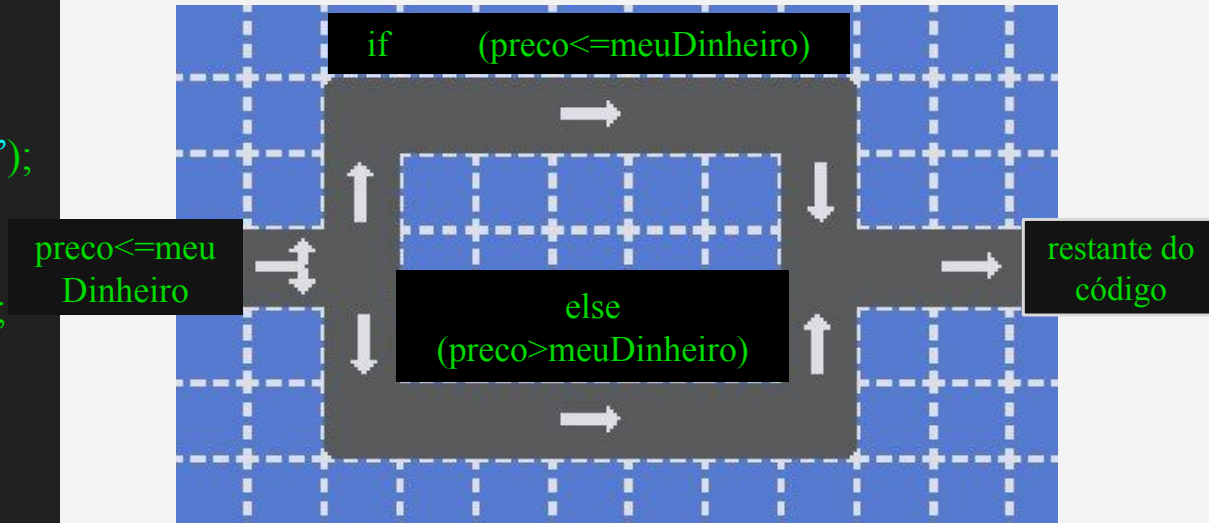


If e else



```
int preco = 20;
int meuDinheiro = 15;

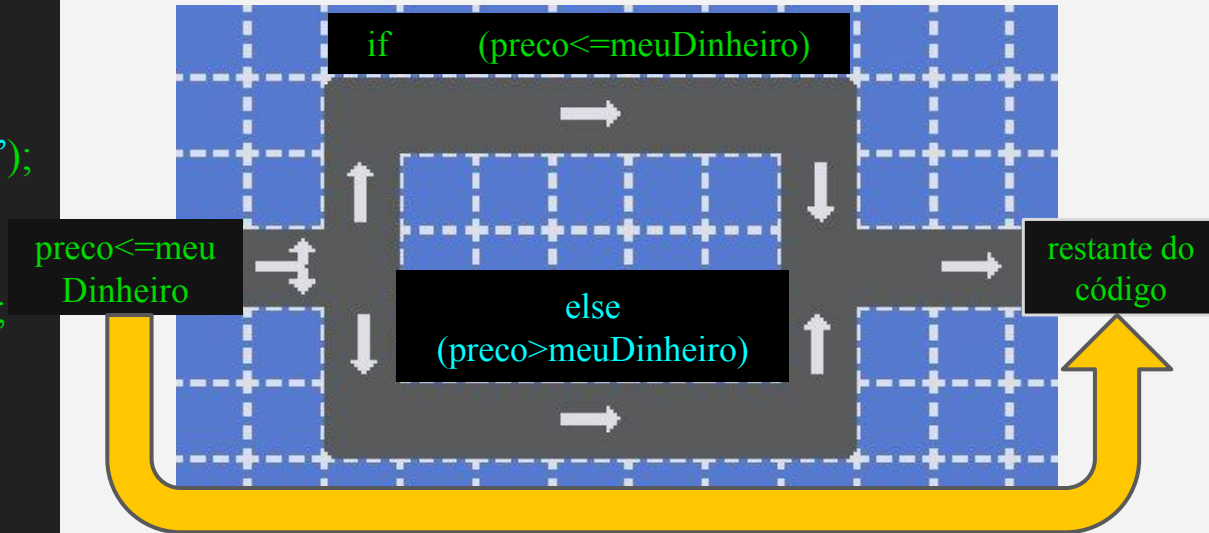
if (preco <= meuDinheiro) {
    System.out.println("ae comprei");
}
else {
    System.out.println("tá caro :( ");
}
```



If e else



```
int preco = 20;  
int meuDinheiro = 15;  
  
if (preco <= meuDinheiro) {  
    System.out.println("ae comprei");  
}  
else {  
    System.out.println("tá caro :( ");  
}
```



Else if



```
if (condição) {  
    executa;  
}  
else if (condição) {  
    executa;  
}  
else {  
    executa;  
}
```

A cláusula *else if* pode
aparecer quantas vezes
for necessário!

// caso as condições do if e dos else if não forem
// cumpridas, executa o que está dentro do else

If



```
int x = 32;

if (x > 50) {
    System.out.println(":(");
}
else if (x > 20) {
    System.out.println(":)");
}
else {
    System.out.println(":/");
}

//imprime: :)
```

```
int x = 62;

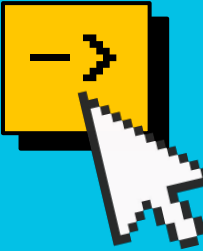
if (x > 50) {
    System.out.println(":(");
}
else if (x > 20) {
    System.out.println(":)");
}
else {
    System.out.println(":/");
}

//imprime: :(
```



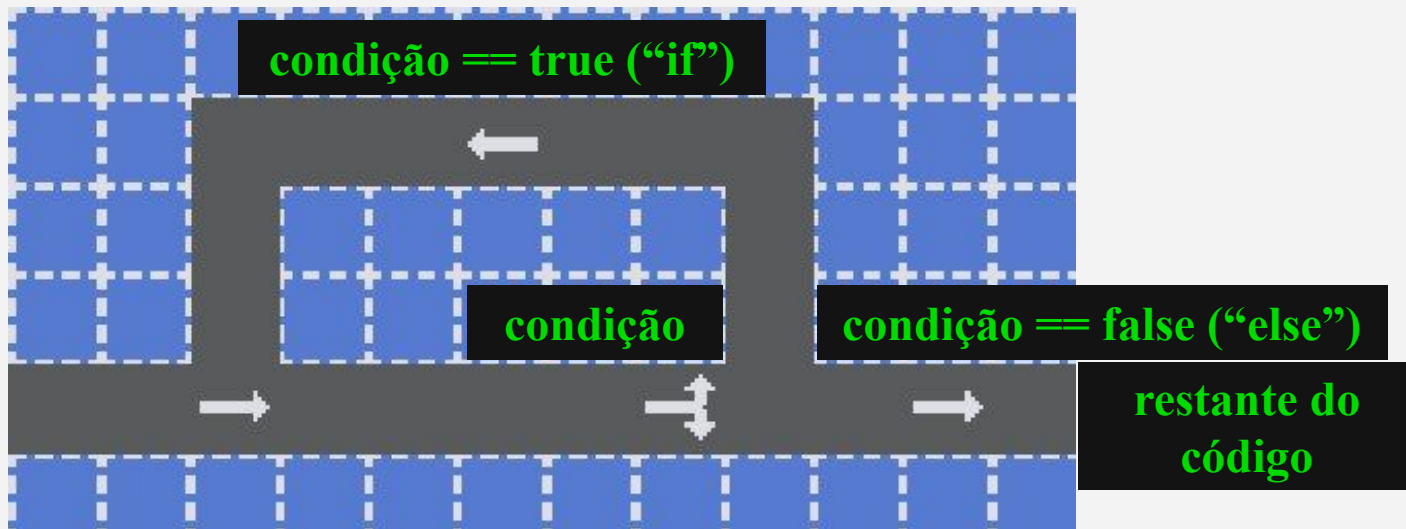

Aula nº03

Loops
(while, for
e do while)



Loops são estruturas de repetição, que repetem uma parte do seu código enquanto uma condição estabelecida estiver sendo cumprida.

Podemos pensar como a repetição de um if:

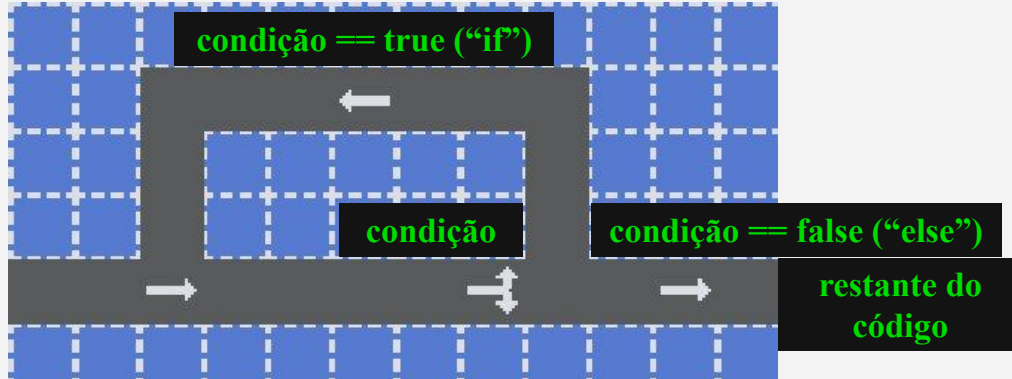


While



O while tem até a estrutura parecida com a do if.

É importante que dentro do código do while tenha alguma lógica que leve a mudança no resultado da condição



```
import java.util.Scanner;

Scanner leitor = new Scanner(System.in);

int preco = 10;
int meuDinheiro = 40;

while (preco <= meuDinheiro) {
    System.out.println("ae comprei");
    meuDinheiro -= preco;
    preco = leitor.nextInt();
}

System.out.println("tá caro :( ");
```

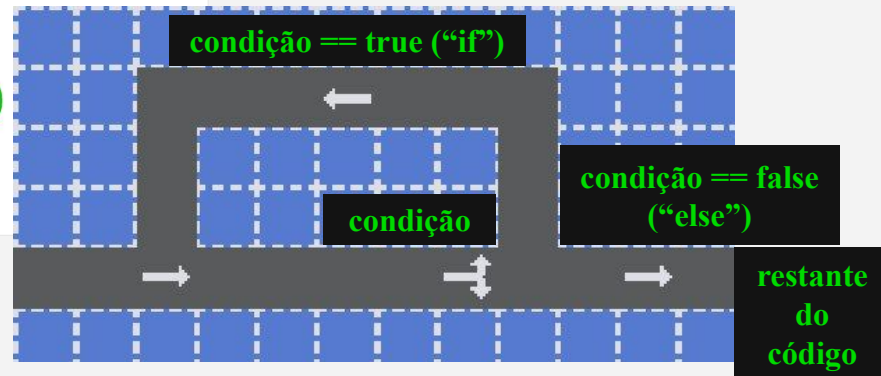
while



```
int x = 0;

while (x < 10) {
    System.out.print(x + " ");
    x++;
}
```

```
//imprime: 0 1 2 3 4 5 6 7 8 9
```



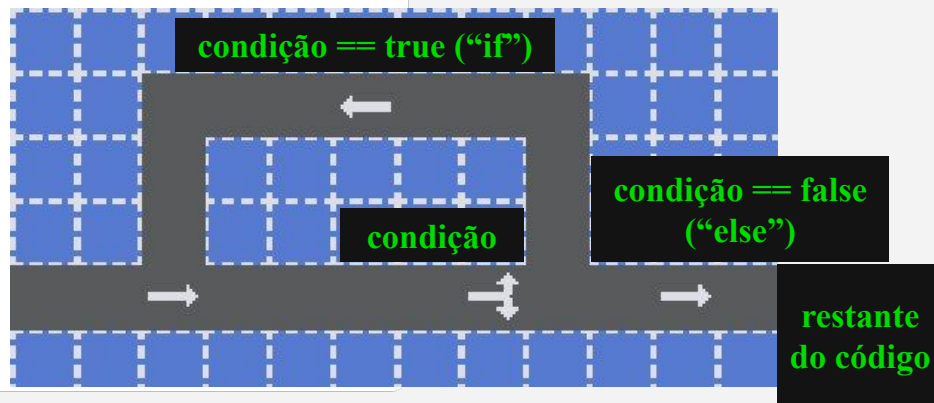
for



```
for (inicialização; condição; atualização) {  
    executa;  
}
```

```
for (int i = 0; i < 10; i++) {  
    System.out.print(i + " ");  
}
```

```
//imprime: 0 1 2 3 4 5 6 7 8 9
```



Podemos usar *break* para sair de um loop e *continue* para pular para a próxima iteração.

Se será utilizado um **while** ou um **for**, cabe ao programador:

- *for* é muito usado quando sabemos a quantidade de repetições antes do loop executar
- já o *while* é mais usado quando a variável da condição do loop já é alterada no código dentro do loop

O *do while* possui um funcionamento muito semelhante ao *while*, porém ele garante que o código dentro do loop **execute ao menos uma vez**, independente da condição.

```
do {  
    executa;  
} while (condição);
```

do while



```
int x = 0;

do {
    System.out.print(x + " ");
    x++;
} while (x < 10);

//imprime: 0 1 2 3 4 5 6 7 8 9
```

```
int x = 10;

do {
    System.out.print(x + " ");
    x++;
} while (x < 10);

//imprime: 10
```


COMPETEC



Obrigado pela
atenção! :)

☐

Don't show this message again

Ok

